From Python Project to Live Website

Django + Git + GitHub + Neon PostgreSQL + Vercel

A step-by-step teaching guide, with the real errors we hit and exactly how each one was fixed.

Instructor: _____ **Batch:** _____

Edition: October 2026. Cloud dashboards change often - always compare screens with the official documentation of Vercel, Neon and GitHub.

Contents

How to Use This Guide 5

Who this is for . . . 5
What you need before starting . . . 5
Colour-coded boxes used throughout . . . 5
Conventions . . . 6

Part 1 - The Big Picture 7

1.1 What actually happens when someone opens your website . . . 7
1.2 The five tools and the job of each . . . 7
1.3 The journey in nine stages . . . 7
1.4 Words every student must know . . . 8

Part 2 - Building the Project From Scratch 9

2.1 Create the project folder . . . 9
2.2 Create and activate a virtual environment . . . 9
2.3 Install packages and record them . . . 9
2.4 Create the Django project and the app . . . 10
2.5 First run . . . 10

Part 3 - Every File and How They Connect 11

3.1 The folder map . . . 11
3.2 How one page request travels through the files . . . 11
3.3 manage.py . . . 12
3.4 config/settings.py - the brain . . . 12
3.5 config/urls.py . . . 14
3.6 config/wsgi.py and asgi.py . . . 14
3.7 portfolio/models.py - the database design . . . 14
3.8 portfolio/migrations/ . . . 15
3.9 portfolio/admin.py . . . 15
3.10 portfolio/views.py - the logic . . . 15
3.11 Templates: base.html, home.html, gallery.html . . . 16
3.12 static/ and media/ . . . 16
3.13 api/main.py - the optional FastAPI service . . . 16
3.14 Configuration and housekeeping files . . . 17

Part 4 - Run and Test Locally First 18

4.1 The commands in order . . . 18

4.2 Automatic tests . . . 18
4.3 Local error reference . . . 18

Part 5 - Git and GitHub: Uploading the Project 19

5.1 The idea in one picture . . . 19
5.2 One-time Git setup . . . 19
5.3 .gitignore - deciding what must never be uploaded . . . 19
5.4 First upload, step by step . . . 19
5.5 The everyday cycle (use this after every change) . . . 20
5.6 Real Git errors from our session . . . 20

Part 6 - Lesson From a First Attempt: Render 22

6.1 What render.yaml says . . . 22
6.2 The error . . . 22
6.3 Why we did not simply reuse that database . . . 22

Part 7 - The Database on Neon 23

7.1 Why the database lives outside Vercel . . . 23
7.2 Create the database . . . 23
7.3 Anatomy of the connection string . . . 23
7.4 How Django uses it . . . 23

Part 8 - Preparing the Project for Vercel 24

8.1 How Vercel runs Django . . . 24
8.2 File 1 - vercel.json . . . 24
8.3 File 2 - .vercelignore . . . 24
8.4 File 3 - the settings.py additions . . . 25
8.5 Optional storage block (explained in Part 11) . . . 25
8.6 Commit and push . . . 25

Part 9 - Deploying on Vercel 26

9.1 Import the repository . . . 26
9.2 Pitfall 1: the wrong preset . . . 26
9.3 Pitfall 2: key and value mixed up . . . 26
9.4 Adding several variables . . . 26
9.5 Watching the build . . . 27
9.6 Verify the live site . . . 27

Part 10 - After Deployment: Admin User and Live Errors 28

10.1 Creating the admin user on the live database . . . 28

10.2 Errors we met while doing this 28
10.3 Errors you may meet on the live site 29

Part 11 - The Upload Problem and File Storage **30**

11.1 Why uploads disappear on Vercel 30
11.2 Three ways to solve it 30
11.3 How the S3 option plugs in 30

Part 12 - Updating, Rolling Back and Maintaining **31**

12.1 The daily workflow 31
12.2 When a deployment breaks 31
12.3 Changing your own information 31
12.4 Custom domain (optional) 31

Part 13 - Master Troubleshooting Table **32**

Part 14 - Security Checklist **33**

Part 15 - Command Glossary **34**

Part 16 - Student Exercises **35**

Appendix A - Final Versions of the Added Files **36**

Appendix B - Suggested Six-Session Plan **37**

Appendix C - Quiz With Answers **38**

How to Use This Guide

This guide follows the exact journey of a real student project: a Django portfolio website that starts as Python files on a Windows laptop and ends as a live website on the internet. Every step is explained three ways: **what to type**, **what it means** and **why it matters**. The real mistakes we made on the way (wrong Git account, hidden dot-files, a missing virtual environment, a database that hung up, and more) are written down as **ERROR** boxes together with the fix, because mistakes are the best teachers.

Who this is for

- Students who know basic Python and have heard of Django, but have never deployed anything.
- Instructors who want a ready lesson plan (see Appendix B) and quiz (Appendix C).
- Anyone who has a Django project and wants to host it for free.

What you need before starting

Item	Why you need it	Where to get it
Windows 10/11 PC	All commands in this guide are shown for PowerShell.	-
Python 3.11 or newer	Runs Django and every script.	python.org or Microsoft Store
Git	Tracks versions and uploads code.	git-scm.com
VS Code (or any editor)	Edit files; shows hidden dot-files.	code.visualstudio.com
GitHub account	Stores the code online; Vercel reads from it.	github.com
Neon account	Free PostgreSQL database in the cloud.	neon.tech
Vercel account	Hosts the live website.	vercel.com

Colour-coded boxes used throughout

WHY IT MATTERS	Explains the reason behind a step, so students learn the idea and not only the keystrokes.
ERROR	**A real error message we saw** Shows the symptom, the cause and the fix.
TIP	A shortcut or a safer way to do the same thing.
SECURITY	Something that can leak a password or break the website if done wrongly.
TEACH	A question or demonstration for the instructor to run in class.

Conventions

- Text in `this style` is typed exactly as shown, or is a file or folder name.
- Words in angle brackets, like `<your-url>`, are placeholders. Replace them, including the brackets.
- Never type real passwords, secret keys or database strings into slides, chats or screenshots. In this guide they are always shown as placeholders.

Part 1 - The Big Picture

1.1 What actually happens when someone opens your website

A website is just a program that answers questions. A visitor's browser asks *"please give me the page at /gallery/"*. Your Django program receives that question, reads data from a database if needed, builds an HTML page and sends it back. Deployment means putting that program on a computer that is switched on all day and reachable from anywhere. We will use free cloud services instead of buying a server.

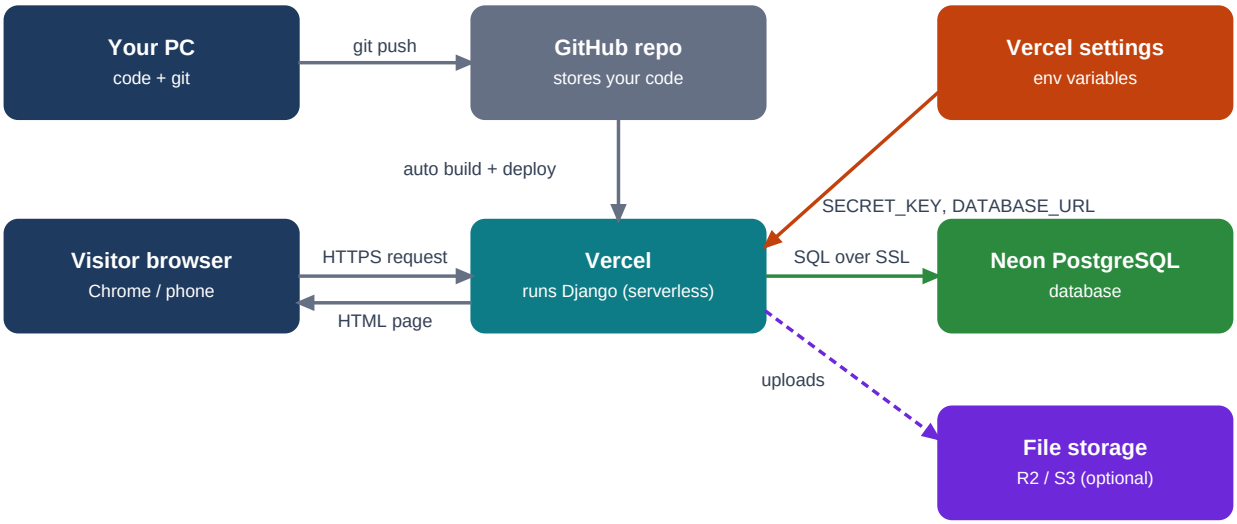


Figure 1: The complete system. Code travels left to right through Git; visitors travel through Vercel to the database.

1.2 The five tools and the job of each

Tool	Job	Analogy
Django (Python)	The program that builds pages, talks to the database, and provides an admin panel.	The restaurant kitchen
Git	Saves every version of your files on your PC so you can go back or compare.	A time machine for files
GitHub	Keeps a copy of the Git history online. Vercel downloads your code from here.	A cloud locker for the kitchen's recipes
Neon	A PostgreSQL database in the cloud. Stores contact messages, gallery records, admin users.	The pantry and the order book
Vercel	Builds your project when you push and runs it for visitors.	The restaurant building with its address

1.3 The journey in nine stages

1. Create the Django project and run it on your own PC.
2. Understand every file and how the files connect.
3. Put the project under Git version control.
4. Upload (push) the project to GitHub.
5. Learn from a first deployment attempt on Render that hit a free-tier limit.
6. Create a PostgreSQL database on Neon.

7. Add three small files and settings changes so Vercel can run the project.
8. Deploy on Vercel, fill in the environment variables and fix what breaks.
9. Create the admin user on the live database, then handle file-upload storage.

1.4 Words every student must know

Term	Meaning in plain language
Framework	A big toolbox of ready-made code. Django handles URLs, databases, forms, security.
Virtual environment (.venv)	A private folder of Python packages for one project, so projects do not disturb each other.
Package / dependency	Code written by others that your project uses, such as Django or Pillow.
Migration	A small file that describes a change to the database tables. Running migrate applies them.
ORM	Lets you work with database rows as Python objects instead of writing SQL by hand.
Static files	Files that never change per user: CSS, JavaScript, logos, the resume PDF.
Media files	Files uploaded by users or admins while the site runs, such as gallery PDFs.
Environment variable	A named setting stored outside the code (like SECRET_KEY) so secrets never go on GitHub.
Repository (repo)	A project folder tracked by Git.
Commit	A saved snapshot of your files with a message.
Push	Upload your commits to GitHub.
Serverless	Your code runs only while a request arrives. There is no permanent disk and nothing stays running.
Connection pooling	A middleman that shares a few database connections among many short-lived requests.
CSRF	A security check that stops other websites from secretly submitting your forms.

Part 2 - Building the Project From Scratch

Even if you are given a finished project, students should build a tiny one once. These are the exact steps that produced the project structure used in this guide.

2.1 Create the project folder

```
mkdir D:\Portfolio\pravin_portfolio
cd D:\Portfolio\pravin_portfolio
```

`mkdir` makes a new folder. `cd` (change directory) moves your terminal into it. Every later command runs from inside this folder, which we call the **project root**.

2.2 Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

- `python -m venv .venv` asks Python to build a private copy of itself inside a folder named `.venv`.
- `Activate.ps1` switches your terminal to use that private copy. You will see `(.venv)` at the start of the prompt.

WHY IT MATTERS	Without a virtual environment every project shares one pile of packages. Project A needs Django 4, project B needs Django 5, and one of them breaks. The virtual environment also keeps the list of packages for deployment accurate.

ERROR	**Running scripts is disabled on this system (PowerShell)** **Cause:** Windows blocks `.ps1` scripts by default. **Fix:** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again. `-Scope Process` means the change lasts only for this window.

ERROR	**No module named 'dj_database_url' (seen later in our class)** **Cause:** The prompt had lost its `(.venv)` prefix, so PowerShell used the system Python, which does not have the project's packages. **Fix:** Activate the environment again (`.\.venv\Scripts\Activate.ps1`) or run `pip install -r requirements.txt`. Always check for `(.venv)` before running Django commands.

2.3 Install packages and record them

```
pip install Django dj-database-url "psycopg[binary]" gunicorn whitenoise Pillow
pip freeze > requirements.txt     (or write the list by hand, as in this project)
```

The project's `requirements.txt` lists what the website needs:

```
Django>=5.2,<5.3          the web framework
dj-database-url>=2.3      turns DATABASE_URL text into Django database settings
psycopg[binary]>=3.2      the driver that lets Python talk to PostgreSQL
gunicorn>=23.0            a production web server (used on Render)
whitenoise>=6.9           serves CSS/JS/images directly from Django
Pillow>=11.0              required by Django for ImageField (gallery cover images)
fastapi>=0.115            used only by the optional api/ folder
uvicorn[standard]>=0.34   server for FastAPI
SQLAlchemy>=2.0           database toolkit used by the FastAPI file
```

WHY IT MATTERS	The hosting service reads `requirements.txt` and installs exactly these packages. If a package is missing here, your live site crashes with ModuleNotFoundError even though it worked on your laptop.

2.4 Create the Django project and the app

```
django-admin startproject config .
python manage.py startapp portfolio
```

- **startproject config .** creates `manage.py` and a folder `config/` holding the settings. The final **dot** means "here", so files land in the current folder instead of a nested one.
- **startapp portfolio** creates an app folder. A Django *project* is the whole site; an *app* is one feature area inside it. Here the single app is called portfolio.
- Register the app by adding `"portfolio"` to `INSTALLED_APPS` in `config/settings.py`. Django ignores apps that are not listed.

2.5 First run

```
python manage.py migrate
python manage.py runserver
```

Open **http://127.0.0.1:8000**. `127.0.0.1` always means "this computer", and 8000 is the port. `migrate` creates the built-in tables (users, sessions, admin log) in a local SQLite file called `db.sqlite3`.

ERROR	**python is not recognized as an internal or external command** **Cause:** Python is not on the PATH. **Fix:** Reinstall Python and tick *Add python.exe to PATH*, or use `py` instead of `python`.

ERROR	**Error: That port is already in use** **Fix:** Stop the other server with Ctrl+C or run `python manage.py runserver 8001`.

TEACH	Ask: "If I delete the .venv folder, have I lost any code?" (No. Only installed packages, which can be rebuilt from requirements.txt. This is why .venv is never uploaded.)

Part 3 - Every File and How They Connect

3.1 The folder map

```
pravin_portfolio/                  <- project root (run every command here)
|-- manage.py                      command-line control panel for Django
|-- requirements.txt               list of packages to install
|-- .env.example                   template of settings (safe to share)
|-- .env                           your real settings (NEVER upload)
|-- .gitignore                     tells Git which files to skip
|-- render.yaml                    blueprint for Render hosting (first attempt)
|-- vercel.json                    build instructions for Vercel   (added later)
|-- .vercelignore                  files Vercel should not upload  (added later)
|-- README.md, FILE_INVENTORY.txt  documentation
|-- db.sqlite3                     local test database (not uploaded)
|-- config/                        the project's settings and wiring
|   |-- settings.py                all configuration
|   |-- urls.py                    URL -> view routing table
|   |-- wsgi.py / asgi.py          entry points for web servers
|-- portfolio/                     the app (the real features)
|   |-- models.py                  database tables as Python classes
|   |-- migrations/                history of table changes
|   |-- admin.py                   how tables appear in /admin/
|   |-- views.py                   the logic for each page
|   |-- apps.py, tests.py          app registration and automatic tests
|-- templates/portfolio/           HTML pages: base, home, gallery
|-- static/                        css, js, images, Resume.pdf
|-- media/                         uploaded gallery files
|-- staticfiles/                   generated by collectstatic (not uploaded)
|-- api/                           optional separate FastAPI service
```

3.2 How one page request travels through the files

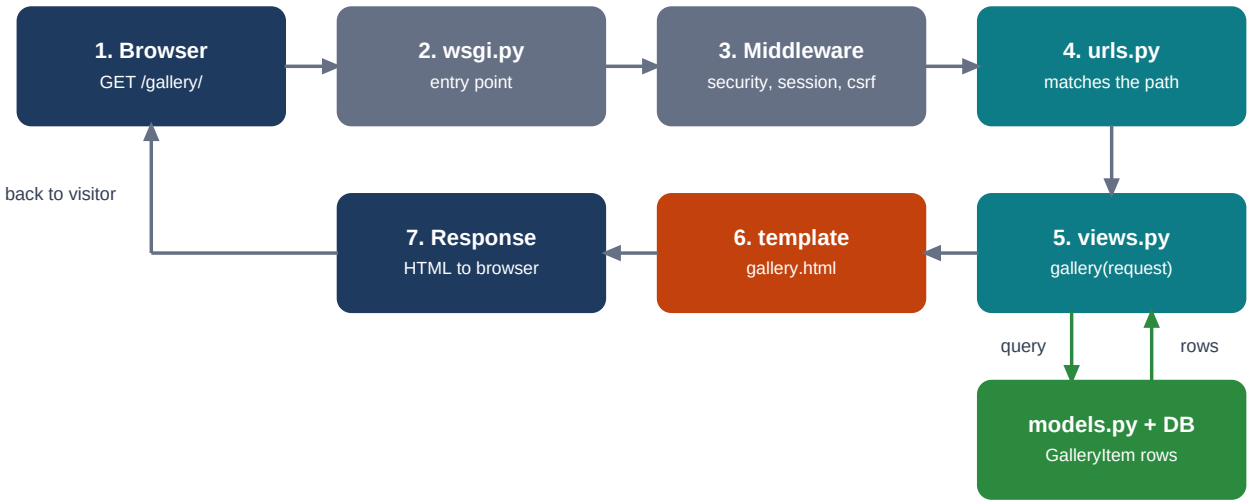


Figure 2: The life of one request, using the Gallery page as the example.

Read Figure 2 as a story. A visitor asks for **/gallery/**.

1. The web server calls the `application` object in `config/wsgi.py`.
2. The request passes through each entry of `MIDDLEWARE` in `settings.py` (security headers, WhiteNoise, sessions, CSRF, login).
3. `config/urls.py` finds `path("gallery/", views.gallery)` and chooses the view.
4. The function `gallery(request)` in `portfolio/views.py` runs.

5. It asks the model `GalleryItem` (in `models.py`) for all public rows. Django turns that into SQL and the database answers.
6. The view hands the rows to `templates/portfolio/gallery.html`, which *extends* `base.html` and loops over the items.
7. The finished HTML is returned to the visitor. The browser then requests CSS, JS and images, which come from `static/` through WhiteNoise.

TEACH	Run the project, open /gallery/, and ask students to point at each file in turn. Then break something on purpose (rename a template) and read the error together.

3.3 manage.py

```
import os, sys
if __name__ == "__main__":
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
    from django.core.management import execute_from_command_line
    execute_from_command_line(sys.argv)
```

This is the command-line control panel. When you type `python manage.py migrate`, the word **migrate** is passed in `sys.argv`. The line with `DJANGO_SETTINGS_MODULE` tells Django *where its settings are*: the file `config/settings.py`. Vercel also finds your project by looking for this file.

3.4 config/settings.py - the brain

Settings are best learned block by block. Here are the blocks from this project with their purpose.

Block 1: reading secrets from the environment

```
SECRET_KEY = os.getenv("SECRET_KEY", "local-only-change-me")
DEBUG = os.getenv("DEBUG", "True").lower() == "true"
ALLOWED_HOSTS = [x.strip() for x in os.getenv("ALLOWED_HOSTS",
                 "127.0.0.1,localhost").split(",") if x.strip()]
CSRF_TRUSTED_ORIGINS = [x.strip() for x in
                 os.getenv("CSRF_TRUSTED_ORIGINS", "").split(",") if x.strip()]
```

- **os.getenv(NAME, default)** reads an environment variable; if it is missing it uses the default. So the same code works on your laptop (defaults) and in production (real values).
- **SECRET_KEY** signs sessions and password-reset links. Anyone who knows it can forge them.
- **DEBUG**: when True, errors show full technical pages. Must be False on a live site.
- **ALLOWED_HOSTS**: which website addresses Django will answer to. Anything else gives "DisallowedHost" (error 400).
- **CSRF_TRUSTED_ORIGINS**: which https origins may submit forms. It must include your exact live URL or form posts fail with "CSRF verification failed".

Block 2: apps and middleware

```
INSTALLED_APPS = ["django.contrib.admin", "django.contrib.auth", ...,
                  "django.contrib.staticfiles", "portfolio"]
MIDDLEWARE = ["django.middleware.security.SecurityMiddleware",
              "whitenoise.middleware.WhiteNoiseMiddleware",
              "django.contrib.sessions.middleware.SessionMiddleware", ...]
```

- **INSTALLED_APPS** switches features on. The last item, `portfolio`, is our own app.
- **MIDDLEWARE** is a chain every request passes through. Order matters: WhiteNoise sits right after the security layer so that static files are answered quickly.

Block 3: URL entry points and templates

```
ROOT_URLCONF = "config.urls"
TEMPLATES = [{"BACKEND": "django.template.backends.django.DjangoTemplates",
              "DIRS": [BASE_DIR / "templates"], "APP_DIRS": True, ...}]
WSGI_APPLICATION = "config.wsgi.application"
```

- `ROOT_URLCONF` points to `config/urls.py`, the routing table.
- `DIRS` tells Django to look for HTML in the top-level `templates/` folder.
- `WSGI_APPLICATION` names the entry point. Vercel reads this exact setting to know how to start your site.

Block 4: the database switch

```
if os.getenv("DATABASE_URL"):
    DATABASES = {"default": dj_database_url.parse(os.environ["DATABASE_URL"],
                 conn_max_age=0 if os.getenv("VERCEL") else 600,
                 ssl_require=not DEBUG)}
else:
    DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3",
                             "NAME": BASE_DIR / "db.sqlite3"}}
```

- If `DATABASE_URL` exists (production) Django uses PostgreSQL; otherwise it uses the local SQLite file. This one block is why the same code runs everywhere.
- `dj_database_url.parse` converts a single text string into the user, password, host, port and database name that Django needs.
- `conn_max_age` is how long a connection is kept open. On Vercel we use 0 because a serverless function may freeze, leaving a dead connection behind.
- `ssl_require` forces encrypted connections when DEBUG is False. Neon requires it.

Block 5: static and media files

```
STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
STATICFILES_DIRS = [BASE_DIR / "static"]
STORAGES = {"default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
            "staticfiles": {"BACKEND":
                "whitenoise.storage.CompressedManifestStaticFilesStorage"}}
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

- **STATICFILES_DIRS** = where *you* keep CSS/JS/images (`static/`).
- **STATIC_ROOT** = where `collectstatic` copies *everything* (yours plus the admin's) into one folder for serving.
- **WhiteNoise** with the "Manifest" storage adds a hash to every file name (for example `style.3fa9c2.css`) so browsers cache safely and refresh when a file changes.
- **MEDIA_ROOT** is where uploaded gallery files are saved. On a normal server this is a real folder; on Vercel it cannot be (see Part 11).

Block 6: cookie and HTTPS security

```
SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
SECURE_SSL_REDIRECT = os.getenv("SECURE_SSL_REDIRECT", "False").lower() == "true"
```

When the site is live (`DEBUG=False`), cookies are marked *secure* so browsers send them only over HTTPS. If you test a DEBUG=False site over plain http locally, login will seem broken. That is expected.

3.5 config/urls.py

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", views.home, name="home"),
    path("contact/", views.contact, name="contact"),
    path("gallery/", views.gallery, name="gallery"),
    path("health/", views.health, name="health"),
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

- Each `path(route, view, name)` says: when the address matches *route*, run *view*. The empty string means the home page.
- The `name` lets templates write `{% url 'gallery' %}` instead of hard-coding an address, so changing the route later does not break links.
- **/health/** returns a tiny JSON reply. Hosting services and humans use it to check that the site is alive.
- The last block serves uploaded media *only when DEBUG is True*. In production a storage service must serve media.

TIP	Clean-up exercise for students: the file imports `path` twice and imports `include` without using it. Remove the unused lines and re-run the tests.

3.6 config/wsgi.py and asgi.py

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
application = get_wsgi_application()
```

WSGI is the standard handshake between a Python web app and a web server. The server imports the variable `application` and calls it for every request. `asgi.py` is the newer asynchronous version; this project uses WSGI.

3.7 portfolio/models.py - the database design

```
class GalleryItem(models.Model):
    KIND_CHOICES = [("image","Image"),("video","Video"),("pdf","PDF"),("note","Note")]
    title = models.CharField(max_length=180)
    description = models.TextField(blank=True)
    kind = models.CharField(max_length=12, choices=KIND_CHOICES, default="pdf")
    file = models.FileField(upload_to="gallery/")
    created_at = models.DateTimeField(auto_now_add=True)
    is_public = models.BooleanField(default=True)
    cover_image = models.ImageField(upload_to='gallery/covers/', blank=True, null=True)

class ContactMessage(models.Model):
    name = models.CharField(max_length=120);  email = models.EmailField()
    subject = models.CharField(max_length=180, blank=True)
    message = models.TextField();  created_at = models.DateTimeField(auto_now_add=True)
```

Each class becomes a database **table**; each attribute becomes a **column**; each saved object becomes a **row**.

Field	Meaning
CharField(max_length)	Short text with a limit.
TextField	Long text, no limit.
FileField(upload_to)	Stores the *path* of an uploaded file in the database and the file itself in storage under the folder given.
ImageField	A FileField that also checks the file is a real image (this is why Pillow is installed).

Field	Meaning
auto_now_add=True	Fill in the current time automatically when the row is first created.
blank=True / null=True	blank: the form may be left empty. null: the database may store NULL.
choices	Limits a value to a list and shows a drop-down in the admin.

3.8 portfolio/migrations/

A migration is a recipe the database follows. After you edit `models.py` you run:

```
python manage.py makemigrations     # writes a new file in migrations/
python manage.py migrate            # applies unapplied migration files to the database
```

- `0001_initial.py` creates the two tables.
- `0002_galleryitem_cover_image.py` adds one column, `cover_image`. Its `dependencies` line says "run 0001 first".
- Django remembers which files were applied in a table named `django_migrations`. That is why you can safely run `migrate` on every deployment; it only applies what is new.

WHY IT MATTERS	Migration files **must be uploaded to GitHub**. If they are missing, the live database never receives the new tables and pages crash with "relation does not exist".

3.9 portfolio/admin.py

```
@admin.register(GalleryItem)
class GalleryItemAdmin(admin.ModelAdmin):
    list_display = ("title", "kind", "is_public", "created_at")
    list_filter = ("kind", "is_public")
    search_fields = ("title", "description")
```

Registering a model gives you a ready-made data-entry screen at **/admin/**. `list_display` chooses the columns, `list_filter` adds the filter sidebar, `search_fields` adds the search box. For `ContactMessage` the fields are `readonly_fields`, so you can read what visitors wrote but not edit it.

3.10 portfolio/views.py - the logic

```
def home(request):
    items = GalleryItem.objects.filter(is_public=True).order_by("-created_at")[:12]
    return render(request, "portfolio/home.html",
                  {"profile": PROFILE, "gallery_items": items})
```

- A **view** is a Python function that receives `request` and returns a response.
- `GalleryItem.objects.filter(is_public=True)` is the ORM: it becomes `SELECT ... WHERE is_public = true`. `"-created_at"` (the minus sign) means newest first; `[:12]` takes only twelve.
- `render(request, template, context)` merges data into HTML. The dictionary is the **context**; its keys become variables inside the template.
- The `PROFILE` dictionary at the top holds your name, skills, jobs and projects as plain Python data. Editing your CV means editing this dictionary, no database needed.

```
@require_http_methods(["GET", "POST"])
def contact(request):
    if request.method == "POST":
        name = request.POST.get("name", "").strip()
        ...
        ContactMessage.objects.create(name=name, email=email, subject=subject, message=message)
        messages.success(request, "Thanks! Your message has been saved.")
```

```
        return redirect("/#contact")
```

- The decorator allows only GET and POST; other methods get error 405.
- The form data arrives in `request.POST`. `.strip()` removes stray spaces.
- `objects.create(...)` inserts a row. This is the code that writes to Neon in production.
- `messages.success` stores a one-time notice that the template prints on the next page.
- `redirect` after a POST prevents a duplicate submission when the visitor refreshes.

3.11 Templates: base.html, home.html, gallery.html

Template tag	Meaning
{% load static %}	Turns on the static tag in this file.
{% static 'css/style.css' %}	Builds the correct URL of a static file, including the cache hash.
{% extends 'portfolio/base.html' %}	This page reuses the outer frame (menu, footer) of base.html.
{% block content %} ... {% endblock %}	The slot in base.html that each page fills with its own content.
{{ item.title }}	Prints a variable. Django escapes dangerous characters automatically.
{% for item in items %} ... {% empty %} ... {% endfor %}	Loop; the empty branch shows when there are no items.
{% url 'contact' %}	Looks up the route named contact in urls.py.
{% csrf_token %}	Inserts a hidden security token into the form. Without it Django rejects the POST.

base.html contains the page skeleton: the head with the CSS link, the top navigation, `<main>{% block content %}{% endblock %}</main>`, the footer and the script tag for `main.js`. **home.html** and **gallery.html** fill that block. Edit the menu once in base.html and every page changes.

3.12 static/ and media/

Path	Role
static/css/style.css	All styling and the dark/light theme.
static/js/main.js	Smooth scrolling, active menu highlight, theme toggle, mobile menu, project pop-up.
static/img/	Room background, profile photo, SVG icons.
static/docs/Resume.pdf	The downloadable resume linked from the top bar.
media/gallery/	Files uploaded through the admin (PDFs, a video, cover images).
staticfiles/	Auto-generated by collectstatic. Never edit it; never commit it.

3.13 api/main.py - the optional FastAPI service

This is a separate small program that returns JSON: `/api/profile`, `/api/projects`, `/api/skills`, `/health`, `/api/db-check`. It is started with `uvicorn api.main:app`. The Django pages **do not call it**, which is why we left it out of the Vercel deployment. It reads the same `DATABASE_URL` and has a CORS list that controls which websites may call it from a browser.

3.14 Configuration and housekeeping files

File	What it does	Upload to GitHub?
.env	Your real local settings and secrets.	**NO**
.env.example	Same names with empty values. Shows teammates what to set.	Yes
.gitignore	List of files Git must skip (.venv, .env, db.sqlite3, staticfiles, caches).	Yes
requirements.txt	Packages to install.	Yes
render.yaml	Render blueprint (see Part 6).	Yes (optional)
vercel.json, .vercelignore	Vercel build rules (Part 8).	Yes
db.sqlite3	Local test data. Real data lives in Neon.	No
README.md	Project description and run instructions.	Yes

Part 4 - Run and Test Locally First

WHY IT MATTERS	Never deploy a project that does not run on your own computer. Local errors are fast to fix; cloud errors cost minutes per attempt.

4.1 The commands in order

```
python -m venv .venv                      # once per project
.\.venv\Scripts\Activate.ps1             # every new terminal window
pip install -r requirements.txt           # install packages
Copy-Item .env.example .env               # create your private settings file
python manage.py migrate                  # create tables in db.sqlite3
python manage.py createsuperuser          # make the admin login
python manage.py runserver                # start at http://127.0.0.1:8000
```

- `Copy-Item` is the PowerShell version of copy. Edit the new `.env` afterwards.
- `createsuperuser` asks for a username, optional email and a password. The password is not shown while you type; that is normal.
- Visit `/admin/`, log in, add a Gallery item with a small PDF, then open `/gallery/`.

4.2 Automatic tests

```
python manage.py test
```

The project includes a smoke test that fetches the home page and checks for status 200 and for the owner's name. Tests are your safety net: run them before each push.

4.3 Local error reference

Message	Cause	Fix
no such table: portfolio_galleryitem	Migrations not applied.	Run `python manage.py migrate`.
TemplateDoesNotExist	Wrong template name or folder.	Check the path inside templates/ and the name in views.py.
DisallowedHost (400)	Address not in ALLOWED_HOSTS.	Add it to the ALLOWED_HOSTS variable.
Missing staticfiles manifest entry	A file referenced by {% static %} does not exist (DEBUG False).	Fix the file name; run collectstatic.
Pillow not installed (ImageField)	Package missing.	pip install Pillow

Part 5 - Git and GitHub: Uploading the Project

5.1 The idea in one picture

```
Working folder --add--> Staging area --commit--> Local history --push--> GitHub
(files you edit)        (files chosen for         (saved snapshots       (online copy;
                         the next snapshot)        on your PC)            Vercel reads it)
```

WHY IT MATTERS	Git does not upload anything until you push. A commit alone is saved only on your computer. Beginners often commit and then wonder why GitHub still shows the old code.

5.2 One-time Git setup

```
git config --global user.name "Your Real Name"
git config --global user.email "you@example.com"
```

Every commit is stamped with this name and email. `--global` applies to all projects on this PC. Leave it out to set the identity for the current project only. Check with `git config user.name` and `git config user.email`.

ERROR	**Accidentally ran git config user.name "Your Name" (copied the example literally)** **Effect:** Only commits made *after* the command carry the wrong name. **Fix:** Run the command again with the real name, then correct the latest commit with `git commit --amend --reset-author --no-edit`. The flag `--reset-author` re-stamps the commit with the current identity and `--no-edit` keeps the message.

5.3 .gitignore - deciding what must never be uploaded

```
.venv/          # thousands of package files, rebuilt from requirements.txt
.env            # SECRETS
.env.*          # any other env variants
!.env.example   # but DO upload the safe template
db.sqlite3      # local test data
__pycache__/    # Python's compiled cache
staticfiles/    # generated by collectstatic
```

SECURITY	A file that was committed once stays in the history even if you later add it to .gitignore. If a secret ever reaches GitHub, treat it as stolen: change the password/key immediately (rotate it), then clean the repo. Prevention is the only easy cure.

5.4 First upload, step by step

1. Create an **empty** repository on github.com (do not tick README or .gitignore, because your folder already has them).
2. In the project root run the commands below.
3. Sign in when the browser window asks. If a password is requested in the terminal, use a **Personal Access Token** instead of your account password.

```
git init                                  # turn the folder into a Git repository
git add .                                 # stage everything not ignored
git status                                # review: green = staged, red = not staged
git commit -m "First version of portfolio" # save the snapshot with a message
git branch -M main                        # name the main branch "main"
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main                   # upload; -u remembers the link for next time
```

Command	Meaning	If you skip it
git init	Creates the hidden .git folder that holds history.	Every other git command says "not a git repository".
git add .	The dot means "everything in this folder".	Commit says "nothing added to commit".
git commit -m	Saves the snapshot; the text after -m describes it.	Nothing is saved, push has nothing to send.
git remote add origin URL	Gives a short name (origin) to the GitHub address.	git push: "no configured push destination".
git push -u origin main	Uploads the main branch; -u links local and remote.	GitHub stays empty.

5.5 The everyday cycle (use this after every change)

```
git status                    # what changed?
git add .                     # or name files: git add vercel.json config/settings.py
git commit -m "Describe the change"
git push                      # Vercel notices and redeploys automatically
```

5.6 Real Git errors from our session

ERROR

fatal: pathspec '.vercelignore' did not match any files
Cause: The file did not exist under that exact name. Our download arrived as `vercelignore` with **no leading dot**. Git matches names exactly.
Diagnose: `dir -Force | findstr vercel` (the `-Force` flag also lists hidden items).
Fix: `Rename-Item vercelignore .vercelignore`.

TIP

Windows can hide dot-files or add .txt to the name. Turn on View - Show - File name extensions in File Explorer, and open the folder in VS Code to see everything.

ERROR

warning: LF will be replaced by CRLF the next time Git touches it
Meaning: Linux/Mac end lines with LF, Windows with CRLF. Git is only telling you it will convert line endings. It is a **warning, not an error**; the commit still succeeds.

ERROR

remote: Permission to A/repo.git denied to B (HTTP 403)
Cause: Windows had saved the login of GitHub account B, but the repository belongs to account A. This happens whenever a student has two GitHub accounts.
Fix, step 1: remove the old saved login: `cmdkey /list | findstr -i github` then `cmdkey /delete:git:https://github.com` (or Control Panel - Credential Manager - Windows Credentials - remove github.com entries).
Fix, step 2: point the project at the right repo: `git remote set-url origin https://github.com/<A>/<repo>.git` and check with `git remote -v`.
Fix, step 3: `git push -u origin main` and sign in as account **A** in the browser (use a private window if the browser is logged in as B).

ERROR

! [rejected] main -> main (non-fast-forward)
Cause: GitHub has commits you do not have (or you rewrote history with --amend).
Fix: If someone else changed GitHub, `git pull --rebase` first. If you are the only user and you intentionally amended a commit: `git push --force-with-lease`. It refuses to overwrite work you have not seen, which makes it safer than plain `--force`.

ERROR	**I committed a secret (.env, a key, a database URL)** **Fix:** Rotate the secret first (new SECRET_KEY, reset the database password in Neon). Then remove the file from tracking with `git rm --cached .env`, make sure it is in .gitignore, and commit. Rotation matters most because the old value remains visible in history.
TEACH	Have each student deliberately make a typo in a file name and read git's error aloud. Fast recognition of "pathspec", "403" and "rejected" saves hours later.

Part 6 - Lesson From a First Attempt: Render

Before Vercel we tried **Render**, a host that runs real servers. The project even includes a `render.yaml` "blueprint". It is worth studying because it shows what a traditional deployment looks like, and why it failed for a reason that has nothing to do with the code.

6.1 What render.yaml says

```
databases:
  - name: pravin-portfolio-db          # create a PostgreSQL database
    plan: free
services:
  - type: web
    name: pravin-portfolio             # the Django site
    buildCommand: pip install -r requirements.txt && python manage.py collectstatic
                  --noinput && python manage.py migrate
    startCommand: gunicorn config.wsgi:application
    envVars: [SECRET_KEY, DEBUG, ALLOWED_HOSTS, DATABASE_URL (from the database), ...]
  - type: web
    name: pravin-portfolio-api         # the FastAPI service
    startCommand: uvicorn api.main:app --host 0.0.0.0 --port $PORT
```

- **buildCommand** runs once at deployment: install packages, gather static files, update the database.
- **startCommand** runs forever afterwards. gunicorn is the production server that calls `application` from wsgi.py.
- **fromDatabase** automatically copies the new database's connection string into DATABASE_URL.

6.2 The error

ERROR	**cannot have more than one active free tier database** **Cause:** The Render account already had a free PostgreSQL database for another project (a bank demo). Free accounts may hold only one, so the Blueprint could not create a second and Render cancelled both web services. **Options:** reuse the existing database, delete an unused one, or pay for another.

6.3 Why we did not simply reuse that database

- **Table clashes.** Both Django projects create tables such as `auth_user` and `django_migrations`. Two projects in one database can corrupt each other's migration history.
- **Safer variant:** create a *second database* inside the same server and give each project its own.
- **Expiry.** Free Render databases expire after a limited time, taking both projects' data with them. Always check the current terms on the provider's pricing page.

WHY IT MATTERS	Lesson: free tiers have limits on number, time and speed. Read them **before** designing a deployment. We moved the database to Neon and the website to Vercel, which also separates the two so that one provider's limit cannot block the other.

Part 7 - The Database on Neon

7.1 Why the database lives outside Vercel

Vercel runs your code as short-lived functions with no permanent disk. A SQLite file would be lost or read-only. The data (contact messages, gallery records, admin users) must live in a database service that stays on all the time. Vercel no longer offers its own Postgres; it connects to partners such as Neon.

7.2 Create the database

1. Go to neon.tech and sign up (signing in with GitHub is easiest).
2. Create a project, e.g. **pravin-portfolio**, and choose a region near you or near your Vercel region.
3. Click **Connect**. A panel shows the connection string.
4. There is a switch called **Connection pooling**. Copy one pooled string (for Vercel) and one non-pooled string (for commands you run from your PC).
5. Store both somewhere private (a password manager), never in chat or screenshots.

7.3 Anatomy of the connection string

```
postgresql://USER:PASSWORD@HOST.neon.tech/DBNAME?sslmode=require
  |            |     |        |                |         |
  protocol   user  secret   server address   database  force encrypted link
pooled host looks like:  ep-xxxx-pooler.region.aws.neon.tech   (note "-pooler")
```

Type	Use it for	Why
Pooled (has -pooler in the host)	The live website on Vercel.	Many short-lived serverless requests share a few real connections, preventing "too many connections".
Direct (no -pooler)	Commands from your PC: migrate, createsuperuser, shell.	Management commands behave more reliably without a pooler in the middle.

SECURITY The connection string contains the password. Anyone who has it can read and delete your data. If it appears in a screenshot, reset the database password in Neon and update Vercel.

7.4 How Django uses it

You never edit database settings by hand for production. You set one environment variable, `DATABASE_URL`, and the block from Part 3.4 does the rest. Locally, leaving it empty falls back to SQLite.

TIP Neon's free compute may suspend after inactivity and wake on the next connection. The very first connection after a long pause can therefore be slow or fail once (see error 10.2). Retry before debugging.

Part 8 - Preparing the Project for Vercel

8.1 How Vercel runs Django

- Vercel detects a Django project by finding **manage.py** and reads the entry point from the `WSGI_APPLICATION` setting. Because of this, no routing rules or special /api folder are required.
- If `STATIC_ROOT` is set, Vercel runs `collectstatic` during the build and serves the static files from its CDN (a network of fast file servers).
- Every page view runs as a **serverless function**: it starts, answers and may be frozen. There is no permanent disk, so files you upload at runtime cannot be kept (Part 11).
- A build command written in `vercel.json` overrides the default, so we include `collectstatic` in it explicitly to be safe.

TEACH	Compare with Render: Render keeps one server running and gunicorn serves requests. Vercel starts your code per request. Ask students which model suits a portfolio with few visitors. (Both do.)

8.2 File 1 - vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": "python manage.py collectstatic --noinput && python manage.py migrate --noinput"
}
```

- `$schema` lets editors validate and auto-complete the file. It has no effect on the site.
- `buildCommand` runs on Vercel's build machine after packages are installed.
- `collectstatic --noinput` gathers all CSS/JS/images into `staticfiles/`; `--noinput` stops it from asking yes/no questions, which would hang an automated build.
- `&&` means "run the next command only if the previous one succeeded".
- `migrate --noinput` creates or updates the tables in Neon using the DATABASE_URL you gave Vercel.

WHY IT MATTERS	Putting migrate in the build means every push also updates the database structure, with no manual step. The risk: a bad migration fails the build, and Vercel keeps serving the previous working version.

8.3 File 2 - .vercelignore

```
.venv/                  local packages (huge, rebuilt by Vercel)
venv/
.git/                   Git history
.env                    secrets
.env.*
db.sqlite3              local test database
*.sqlite3
__pycache__/
api/                    FastAPI folder (not used by the Django site)
render.yaml             Render blueprint (irrelevant here)
FILE_INVENTORY.txt
```

It works like .gitignore but for the upload to Vercel. It keeps the bundle small and stops the `api/` folder from being mistaken for extra serverless functions.

ERROR	**Dot-file named wrongly (see 5.6)** Name it exactly `.vercelignore`, not `vercelignore` and not `.vercelignore.txt`.

8.4 File 3 - the settings.py additions

```
if os.getenv("VERCEL"):                       # Vercel sets this variable automatically
    ALLOWED_HOSTS.append(".vercel.app")       # accept any *.vercel.app address
...
conn_max_age=0 if os.getenv("VERCEL") else 600     # no kept-open DB connections

SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
```

- **ALLOWED_HOSTS**: the leading dot means "this domain and all its sub-domains".
- **conn_max_age** 0: open a fresh connection per request. Safe for serverless and for pooled Neon strings.
- **SECURE_PROXY_SSL_HEADER**: Vercel terminates HTTPS in front of your function. This line tells Django to trust the forwarded header, so it knows the visitor used https. Without it, secure cookies and CSRF checks misbehave.

8.5 Optional storage block (explained in Part 11)

```
if os.getenv("AWS_STORAGE_BUCKET_NAME"):
    STORAGES["default"] = {"BACKEND": "storages.backends.s3.S3Storage"}
    AWS_STORAGE_BUCKET_NAME = os.environ["AWS_STORAGE_BUCKET_NAME"]
    ...
```

The block is dormant until the bucket variable exists, so the same settings file keeps working locally and before you set up storage.

8.6 Commit and push

```
git add vercel.json .vercelignore config/settings.py
git commit -m "Add Vercel config"
git push
```

Always name the files you changed instead of using `git add .` when you want a precise commit.

Part 9 - Deploying on Vercel

9.1 Import the repository

1. Sign in at vercel.com **with the same GitHub account that owns the repo**.
2. Click **Add New - Project** and pick the repository. If it is not listed, choose *Adjust GitHub App Permissions* and grant access.
3. Leave **Root Directory** as ./ because manage.py is at the top.
4. Check the **Application Preset** (next section).
5. Open **Environment Variables**, add the values below, then click **Deploy**.

9.2 Pitfall 1: the wrong preset

ERROR	**Application Preset shows FastAPI** **Cause:** The repository contains an `api/` folder with FastAPI code, so Vercel guessed FastAPI. **Fix:** Change the preset to **Django** (or **Other** so that Vercel detects Django from manage.py). Using the FastAPI preset would try to run the wrong program.

9.3 Pitfall 2: key and value mixed up

ERROR	**The secret was typed into the Key box and Value stayed empty** **Cause:** Two similar boxes. The **Key** is the variable's *name*; the **Value** is its content. **Fix:** Key = `SECRET_KEY`, Value = the long random string. Because the secret had been visible in a screenshot, generate a new one:

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

9.4 Adding several variables

The form shows one row. Click **Add More** for each extra variable, or paste lines of the form `NAME=value` into the Key box, or use **Import .env** with a file holding the lines.

Variable	Example value	What happens without it
SECRET_KEY	long random string	Falls back to an insecure public default. Never leave unset.
DEBUG	False	Defaults to True: visitors can see detailed error pages with internal information.
DATABASE_URL	Neon pooled string	Falls back to SQLite, which cannot persist on Vercel. Forms and admin fail.
CSRF_TRUSTED_ORIGINS	https://your-project.vercel.app	Form posts and admin login fail with "CSRF verification failed".
AWS_* variables	see Part 11	Uploads are not stored permanently.

Leave the environment ticks on **Production and Preview**. The build can succeed without CSRF_TRUSTED_ORIGINS; add it after you see your real address, then redeploy.

9.5 Watching the build

1. Vercel clones your repo, installs `requirements.txt`.
2. Runs the `buildCommand` (collectstatic, migrate).
3. Packages the Django function and uploads static files to the CDN.
4. Prints the live address (for example `https://pravin-portfolio.vercel.app`).

If a step fails, open **Deployments - the failed deployment - Build Logs** and read the **last red lines**; the real cause is almost always there. For runtime crashes use **Runtime Logs**.

9.6 Verify the live site

- Home page loads with styling and your photo.
- `/health/` returns {"status": "ok", ...}.
- Submit the contact form; the success message appears.
- **Redeploying after changing a variable:** Deployments - menu on the latest row - Redeploy. Variables apply only to builds made after you save them.

Part 10 - After Deployment: Admin User and Live Errors

10.1 Creating the admin user on the live database

The live database starts empty of users. You create the first user from your own PC but pointed at Neon:

```
.\.venv\Scripts\Activate.ps1
$env:DATABASE_URL="<neon-string-in-quotes>"
$env:DEBUG="False"
python manage.py createsuperuser
```

- `$env:NAME="value"` sets an environment variable for the **current PowerShell window only**. Nothing is written to disk, which is safer than editing a file.
- Django now reads DATABASE_URL, connects to Neon and inserts the user into the same database the website uses.
- Then log in at `https://<your-url>/admin/` with exactly the username you typed (usernames are case-sensitive).

10.2 Errors we met while doing this

ERROR	**ModuleNotFoundError: No module named 'dj_database_url'** **Cause:** Virtual environment not active (the prompt had no (.venv)). **Fix:** `.\.venv\Scripts\Activate.ps1` then run the command again. If PowerShell blocks scripts: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.
ERROR	**psycopg.OperationalError: consuming input failed: server closed the connection unexpectedly** **Meaning:** The database server hung up on the first query. The build on Vercel had already run migrate successfully, so the string itself was valid; the problem was between the PC and Neon. **Likely causes and fixes, in order:** 1. The database was asleep. Run the same command again. 2. A pooled string was used for a management command. Turn **Connection pooling off** in Neon, copy the direct string, and use it just for your local command (keep the pooled one on Vercel). 3. A network or VPN is blocking the connection. Test with a phone hotspot or switch the VPN off. Quick isolation test: `python -c "import psycopg,os; c=psycopg.connect(os.environ['DATABASE_URL']); print(c.execute('select 1').fetchone())"` prints (1,) when the link works.
TIP	**Fallback with no local connection:** set DJANGO_SUPERUSER_USERNAME, DJANGO_SUPERUSER_EMAIL and DJANGO_SUPERUSER_PASSWORD in Vercel and append `&& (python manage.py createsuperuser --noinput || true)` to the buildCommand. After the first successful deploy remove the password variable and that part of the command.

10.3 Errors you may meet on the live site

Symptom	Cause	Fix
CSRF verification failed	CSRF_TRUSTED_ORIGINS missing or different from the address bar.	Copy the exact https address, no trailing slash, redeploy.
500 Server Error	Missing variable, bad migration or a code exception.	Vercel - Runtime Logs; read the last traceback line.
Page works but looks unstyled	collectstatic did not run or failed.	Check Build Logs for collectstatic; keep the command in vercel.json.
DisallowedHost	Address not in ALLOWED_HOSTS.	Make sure the VERCEL block exists, or add the domain to ALLOWED_HOSTS.
relation "portfolio_..." does not exist	migrate did not reach Neon.	Confirm DATABASE_URL is set for Production, redeploy, read the migrate lines in the log.
Gallery items show but files give 404	Uploads are not stored permanently.	Set up storage (Part 11).

Part 11 - The Upload Problem and File Storage

11.1 Why uploads disappear on Vercel

In the admin you upload a PDF. Django saves it in `MEDIA_ROOT` (the media/ folder). On Vercel the function's disk is temporary and the deployment is read-only, so the file is lost when the function stops, and other function instances never see it. Also, `urls.py` serves media only when DEBUG is True. The database row survives, but the file it points to is gone.

11.2 Three ways to solve it

Option	How	Good for	Cost
S3-compatible bucket (Cloudflare R2, Backblaze B2, Supabase Storage, AWS S3)	django-storages; set AWS_* variables	Real admin uploads, any file size within limits	Free tiers exist; check current limits
Cloudinary	Dedicated Django package	Images and video with automatic resizing	Free tier exists
Put files in static/ and push with Git	No code change; link with {% static %}	A handful of fixed documents	Free; no admin upload

11.3 How the S3 option plugs in

1. Add `django-storages[s3]` and `boto3` to requirements.txt.
2. Create a bucket at your provider and make files publicly readable (or use a public domain).
3. Create an access key and secret for the bucket.
4. Add to Vercel: `AWS_STORAGE_BUCKET_NAME`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_S3_ENDPOINT_URL` (and optionally `AWS_S3_CUSTOM_DOMAIN`).
5. Redeploy, log in to /admin/ and upload the gallery items again. `file.url` in templates now points to the bucket.

WHY IT MATTERS	No template or view changes are needed. Django's storage layer is a plug-in point: `item.file.url` asks the configured storage for the address, whether that is a local folder or a cloud bucket.

SECURITY	Never put the access key in the code or on GitHub. Only in Vercel's environment variables.

Part 12 - Updating, Rolling Back and Maintaining

12.1 The daily workflow

```
(edit files in VS Code)
python manage.py runserver        # look at it locally
python manage.py test             # run the checks
git add <files>
git commit -m "What I changed"
git push                          # Vercel builds and publishes automatically
```

Each push creates a new deployment. Pushes to other branches create **Preview** deployments with their own temporary address, handy for testing before merging into main.

12.2 When a deployment breaks

- Failed **build**: your live site is untouched; the previous deployment stays online. Fix and push again.
- Successful build but broken **behaviour**: open the Deployments list, choose an earlier healthy deployment and promote it (instant rollback) while you repair the code.
- Use `git log --oneline` to see past commits, and `git revert <id>` to undo one safely with a new commit.

12.3 Changing your own information

- Name, skills, jobs, projects: edit the `PROFILE` dictionary in `portfolio/views.py`.
- Design: `static/css/style.css`. Menu and footer: `templates/portfolio/base.html`.
- Resume: replace `static/docs/Resume.pdf` and push.
- New database column: edit models.py, run makemigrations, test locally, commit the new migration file, push.

12.4 Custom domain (optional)

In Vercel open Project - Settings - Domains, add your domain and follow the DNS instructions. Then add the new https address to `CSRF_TRUSTED_ORIGINS` (comma-separated) and redeploy.

Part 13 - Master Troubleshooting Table

Where	Message or symptom	Cause	Fix
PowerShell	running scripts is disabled	Execution policy	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PowerShell	python is not recognized	Python not on PATH	Reinstall with Add to PATH, or use py
Python	No module named X	.venv not active or package missing	Activate .venv; pip install -r requirements.txt
Django	no such table	migrate not run	python manage.py migrate
Django	CSRF verification failed	Origin not trusted	Set CSRF_TRUSTED_ORIGINS to the exact https address
Django	DisallowedHost	Host not allowed	Add domain to ALLOWED_HOSTS
Git	pathspec did not match	Wrong or missing file name	dir -Force; rename (for example add the leading dot)
Git	403 Permission denied to (another user)	Wrong saved account	Delete Windows credential; set-url origin; sign in as owner
Git	rejected non-fast-forward	History differs	git pull --rebase, or push --force-with-lease if you amended
Git	LF will be replaced by CRLF	Line-ending notice	Ignore; it is only a warning
Render	more than one active free tier database	Free limit	Reuse, delete, pay, or move the database (we chose Neon)
Neon	server closed the connection unexpectedly	Asleep, pooler or network	Retry; use direct string locally; check VPN
Vercel	Preset says FastAPI	api/ folder detected	Choose Django or Other
Vercel	Secret typed in Key box	Key/Value confusion	Key = name, Value = content; regenerate the secret
Vercel	No place to add DEBUG, DATABASE_URL	Form shows one row	Click Add More or paste NAME=value lines
Vercel	Build failed	See last red lines	Fix the named file or variable, push again
Vercel	Unstyled live site	collectstatic missing	Keep it in buildCommand
Vercel	Uploaded file is 404	No permanent disk	Use an S3-compatible bucket

Part 14 - Security Checklist

Check	Why
DEBUG is False in Vercel.	Debug pages reveal code paths, settings names and sometimes data.
SECRET_KEY is long, random and only in Vercel.	It signs sessions; leaked keys allow forged sessions.
No .env, .sqlite3 or database URL on GitHub.	Public repos are scraped by bots within minutes.
A secret ever shown in a screenshot or chat is replaced.	Treat it as public the moment it is seen.
Admin password is strong and unique.	/admin/ is a known address that bots probe.
Database user and bucket keys use the least privilege possible.	Limits damage if one leaks.
Dependencies are kept updated (pip list --outdated).	Security fixes arrive in new versions.
Use https only (Vercel does this automatically).	Prevents password sniffing on public Wi-Fi.

Part 15 - Command Glossary

Command	Meaning
mkdir, cd, dir -Force	Make folder; move into folder; list files including hidden ones.
python -m venv .venv	Create a private Python environment in the folder .venv.
.\.venv\Scripts\Activate.ps1	Switch the terminal to that environment.
pip install -r requirements.txt	Install every package listed in the file.
django-admin startproject config .	Create a project whose settings folder is config, here.
python manage.py startapp portfolio	Create an app named portfolio.
makemigrations	Detect model changes and write a migration file.
migrate	Apply migration files to the configured database.
createsuperuser	Create an administrator who can log in to /admin/.
runserver	Start the development server on port 8000.
collectstatic --noinput	Copy all static files into STATIC_ROOT without prompts.
test	Run the automatic tests.
git init / add / commit / push	Create repo; stage files; save snapshot; upload.
git status / log --oneline	Show changes; show short history.
git remote -v / set-url	Show or change the GitHub address.
git commit --amend --reset-author --no-edit	Fix the latest commit's author without changing its message.
git push --force-with-lease	Overwrite GitHub only if no one else added commits.
cmdkey /list, /delete:	Show or remove saved Windows logins.
$env:NAME="value"	Set an environment variable in this PowerShell window only.
python -c "..."	Run one line of Python without a file.

Part 16 - Student Exercises

1. Draw Figure 2 from memory for the /contact/ page (a POST). Which extra step happens in views.py?
2. Add a new field `link` to `GalleryItem`, create the migration and show it in the admin.
3. Add a route `/about/` with its own view and template that extends base.html.
4. Clean up urls.py (duplicate and unused imports) and run the tests.
5. Break the site on purpose three ways (wrong template name, missing migration, DEBUG=False without ALLOWED_HOSTS). Record the error text and the fix for each.
6. Make a second GitHub account, push the project there and reproduce the 403 error, then fix it.
7. Create a second Neon project and deploy a Preview branch that uses it.
8. Add a test that posts to /contact/ and checks that a ContactMessage row exists.
9. Write down every environment variable the project reads and what happens if each is missing.

TEACH	Mini-project: students deploy a one-page Django site of their own using only this guide. Grade on a working URL, a clean Git history and a written list of the errors they met.

Appendix A - Final Versions of the Added Files

vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": "python manage.py collectstatic --noinput && python manage.py migrate --noinput"
}
```

.vercelignore

```
.venv/
venv/
.git/
.env
.env.*
db.sqlite3
*.sqlite3
__pycache__/
api/
render.yaml
FILE_INVENTORY.txt
```

Additions at the end of config/settings.py

```
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")

if os.getenv("AWS_STORAGE_BUCKET_NAME"):
    STORAGES["default"] = {"BACKEND": "storages.backends.s3.S3Storage"}
    AWS_STORAGE_BUCKET_NAME = os.environ["AWS_STORAGE_BUCKET_NAME"]
    AWS_ACCESS_KEY_ID = os.getenv("AWS_ACCESS_KEY_ID")
    AWS_SECRET_ACCESS_KEY = os.getenv("AWS_SECRET_ACCESS_KEY")
    AWS_S3_ENDPOINT_URL = os.getenv("AWS_S3_ENDPOINT_URL") or None
    AWS_S3_REGION_NAME = os.getenv("AWS_S3_REGION_NAME") or None
    AWS_S3_CUSTOM_DOMAIN = os.getenv("AWS_S3_CUSTOM_DOMAIN") or None
    AWS_QUERYSTRING_AUTH = False
    AWS_DEFAULT_ACL = None
```

Also near the top: `if os.getenv("VERCEL"): ALLOWED_HOSTS.append(".vercel.app")` and in the database block `conn_max_age=0 if os.getenv("VERCEL") else 600`.

Appendix B - Suggested Six-Session Plan

Session	Topic	Hands-on outcome
1	Big picture, virtual environment, new Django project (Parts 1-2)	Site runs on 127.0.0.1
2	File tour and request flow, models, admin (Parts 3-4)	Gallery item added through the admin
3	Git and GitHub, .gitignore, common Git errors (Part 5)	Project visible on GitHub
4	Hosting limits, Render lesson, Neon database (Parts 6-7)	Neon project and connection strings
5	Vercel files, environment variables, first deploy (Parts 8-9)	Live URL
6	Admin on live DB, storage, rollback, troubleshooting drill (Parts 10-13)	Admin login on live site; error log written

Appendix C - Quiz With Answers

#	Question	Answer
1	Why do we not upload the .venv folder?	It is huge and machine-specific; it is rebuilt from requirements.txt.
2	What does the dot in 'django-admin startproject config .' do?	Creates the project in the current folder instead of a nested one.
3	Which file connects a URL to a view?	config/urls.py.
4	What is the difference between makemigrations and migrate?	makemigrations writes migration files from model changes; migrate applies them to the database.
5	Why is DEBUG=False required on a live site?	Debug pages expose internal details; secure cookie settings also depend on it.
6	A commit is made but GitHub shows old code. Why?	The commit was not pushed.
7	What caused the 403 permission error when pushing?	A saved login of a different GitHub account; fix by removing the credential and signing in as the repo owner.
8	Which Neon string do you use on Vercel and which on your PC for management commands?	Pooled on Vercel; direct (non-pooled) locally.
9	Why does Vercel need the WSGI_APPLICATION setting?	It reads it to find the entry point of the Django app.
10	Why do uploaded files vanish on Vercel?	The function filesystem is temporary and read-only; use external storage.
11	What is the Key and what is the Value in an environment variable?	Key is the variable's name; Value is its content.
12	After changing an environment variable, what must you do?	Redeploy so a new build uses it.

End of guide. Keep this document with your project and add every new error you meet, together with its fix, to your own troubleshooting table.